SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CROSS-UTILITY SMART CAMPUS RESOURCE MONITOR: INTEGRATED ENERGY COMPETITION AND WATER LEAKAGE DETECTION

Supervisors

DR. C. SIVASANKAR

Presented by

I. MD IMTHIAZ(192521360)

G SAI DEEPAK(19231142)

CSA0925 – Programming in Java

REVIEW-1 FEEDBACK

Engineer to Excel

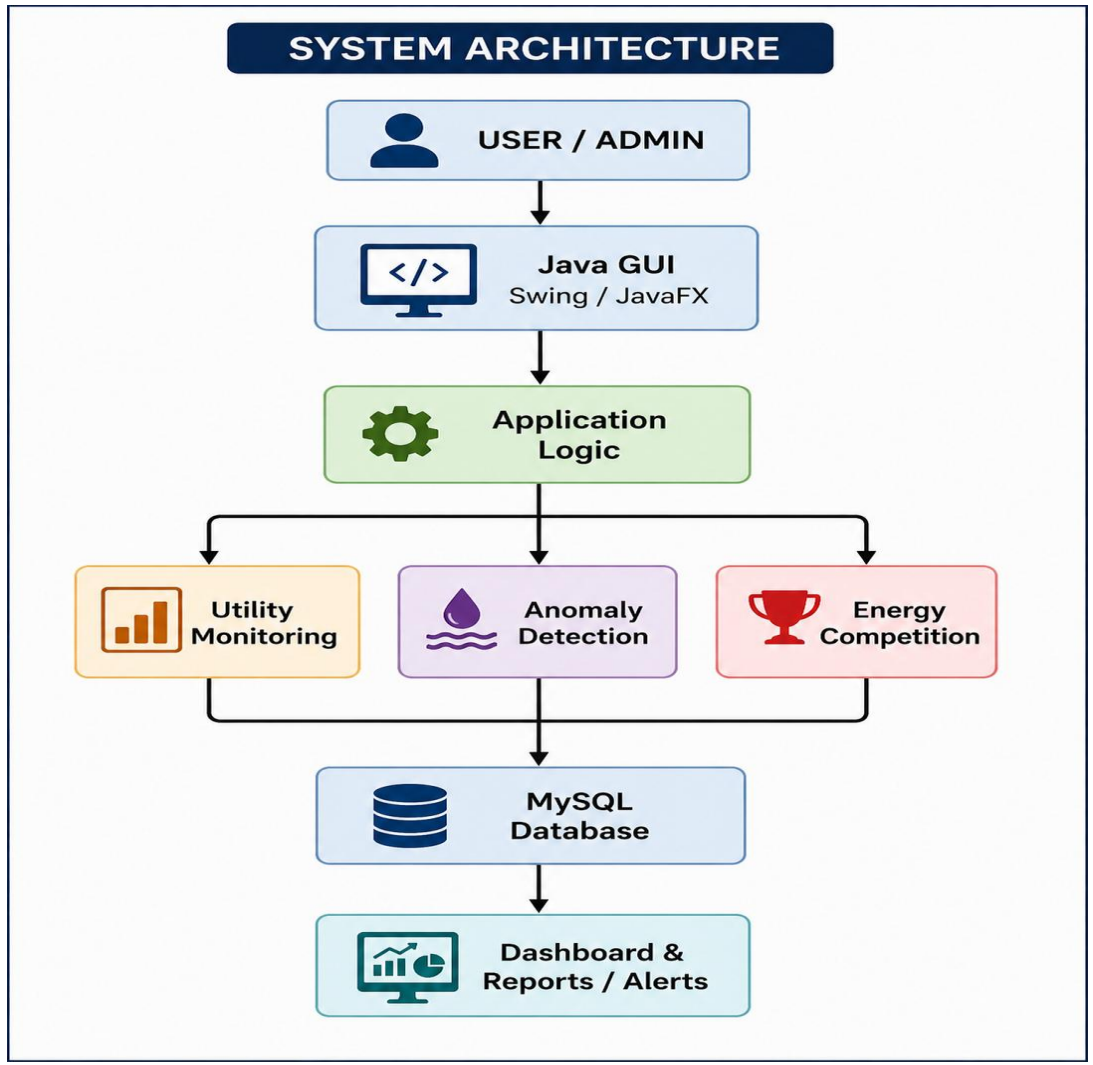
Feedback Received

- Improve system architecture clarity
- Clearly define the project modules
- Include UML and database design
- Improve UI design
- Plan proper testing

Corrective Actions

- Finalized the three-module architecture
- Refined module responsibilities and data flow
- Prepared UML and database design
- Improved dashboard and reporting concepts
- Planned unit and integration testing
- Progressed Java implementation

UPDATED ARCHITECTURE



Changes Made

- Refined the architecture around the three finalized modules
- Clarified data flow from entry to dashboard
- Strengthened database and alert integration

Final Module Flow

User → GUI → Utility Monitoring → Leak Detection → Energy Competition → Database → Dashboard

1. Utility Monitoring Module

- Records electricity and water usage.

2. Leak Detection & Anomaly Alert Module

- Analyze water-usage patterns.
- Detects abnormal consumption.
- Generates alerts.

3. Gamified Energy Competition Module

- Calculates energy efficiency.
- Ranks hostel blocks.
- Displays leaderboard and rewards.

MODULE DESIGN

Engineer to Excel

1. Utility Monitoring Module

- Record electricity and water usage
- Maintain centralized utility data
- Provide monitoring information

2. Leak Detection & Anomaly Alert Module

- Analyze water usage patterns
- Detect abnormal water-flow
- Generate anomaly alerts

3. Gamified Energy Competition Module

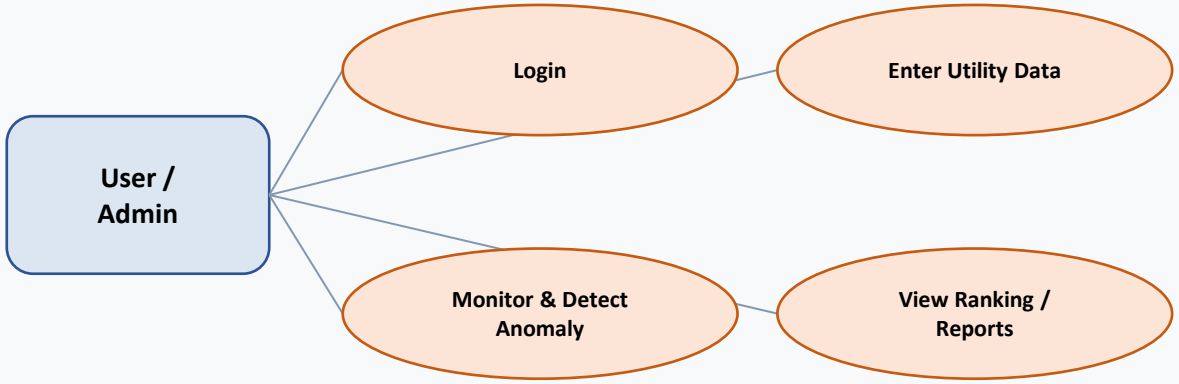
- Calculate energy efficiency
- Compare hostel performance
- Generate rankings/leaderboard

Data Flow & Interfaces

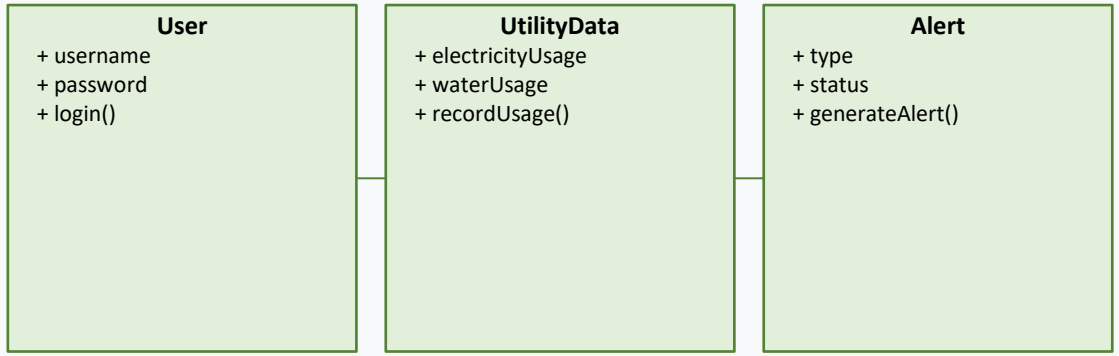
- Data Flow: Data Entry → Database → Processing → Analysis → Alerts/Ranking → Dashboard/Reports
- Interfaces: Module 1 shares utility data via the database; Module 2 reads water data and raises alerts; Module 3 reads energy data for ranking; all modules feed the dashboard

UML & DATABASE

Use Case Diagram

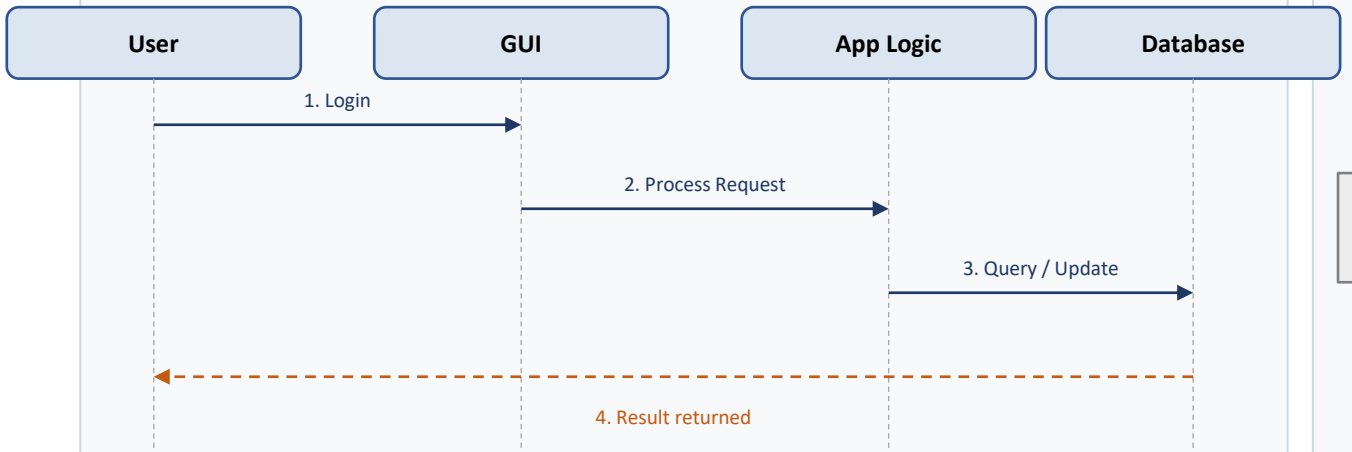


Class Diagram

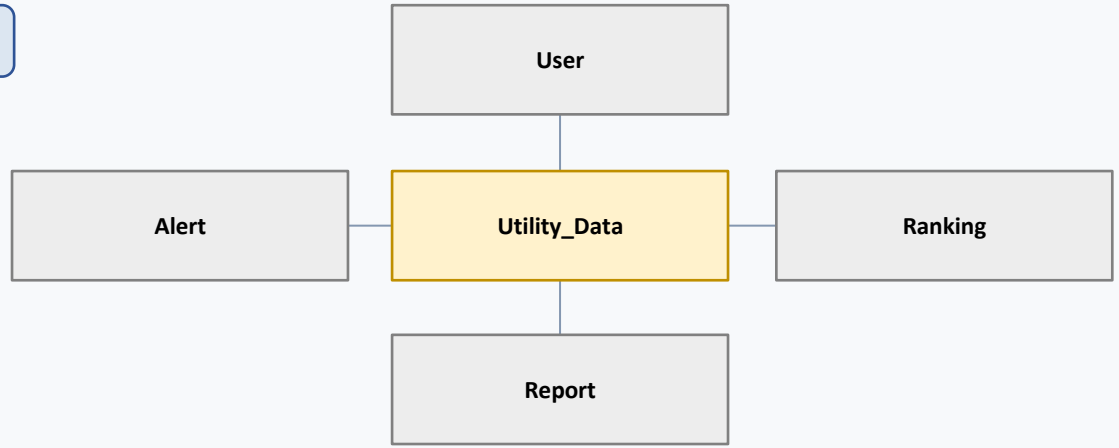


EnergyCompetition and Report classes follow the same structure

Sequence Diagram



ER Diagram



TECHNOLOGY STACK

Programming Language

- Java

Libraries

- Java Collections
- Java I/O

GUI & Database

- GUI: Java Swing / JavaFX
- Database: MySQL
- Connectivity: JDBC

IDE & Hardware

- IDE: IntelliJ IDEA / Eclipse / NetBeans
- Hardware: Not required for the current software prototype

IMPLEMENTATION

Completion %

Code Summary

- Object-oriented Java implementation
- Modular classes per feature
- Data processing logic
- JDBC / database connectivity
- Input validation
- Exception handling

Screenshots

Login / Dashboard screenshot (to be added)

Reports screenshot (to be added)

USER INTERFACE (UI)

Login

Username

Password

Login

Authenticates user / admin credentials

Dashboard

Energy Usage

Water Usage

Alerts

Hostel Ranking

Centralized view of utility usage, alerts and rankings

Reports

Utility Usage Report

Water Anomaly Report

Alert History

Energy-Efficiency Ranking

TESTING

Unit Tests

- Login validation
- Utility data processing
- Water anomaly detection
- Energy-efficiency calculation
- Alert generation

Integration Tests

- Login → Dashboard
- Utility Monitoring → Database
- Water Monitoring → Anomaly Detection
- Anomaly Detection → Alert
- Energy Monitoring → Ranking
- System → Reports

Outputs

- Correct login validation results for valid/invalid credentials
- Accurate anomaly flags for abnormal water-flow readings
- Correct hostel energy-efficiency ranking on the leaderboard

CHALLENGES

Engineer to Excel

Issues

- Designing a clear three-module architecture
- Database connectivity
- Utility data processing
- Water anomaly detection logic
- Maintaining proper data flow

Solutions

- Used modular Java architecture
- Implemented structured data processing
- Planned JDBC database connectivity
- Added validation and exception handling
- Used modular testing

Next Plan

- Complete remaining Java implementation
- Complete database integration
- Complete UI
- Perform full system testing
- Validate outputs
- Finalize documentation
- Prepare final demonstration


thank
you